ORGNZ-03: Bucket Strategy and Design

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Overview

A well-defined bucket strategy optimizes query performance, controls costs, and ensures compliance. This notebook covers naming conventions, retention planning, organizational alignment, and common bucket design patterns.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:bucket-definitions:read`, `storage:logs:read`
Knowledge	Completed ORGNZ-02 (Understanding Grail Buckets)
Data	At least 1 hour of log data

Table of Contents

1. Bucket Naming Conventions
2. Retention Strategy
3. Bucket Design Patterns
4. Cost Control and Attribution
5. Analyzing Bucket Usage

6. Bucket Strategy Considerations
7. Routing Data to Buckets
8. Bucket Design Checklist
9. Query Billing Model

Learning Objectives

By the end of this notebook, you will:
- Design effective bucket naming conventions
- Plan retention strategies for different use cases
- Understand cost implications of bucket design
- Apply organizational alignment patterns

Bucket Naming Conventions

Recommended Naming Patterns

Pattern	Format	Example	Use Case
Provider-based	`<provider>_<type>_<retention>`	`aws_logs_35d`	Multi-cloud environments
LOB-based	`<provider>_<type>_<lob>`	`aws_logs_finance`	Line-of-business retention or access
Team-based	`team_<name>_<type>`	`team_platform_logs`	Team ownership and billing
Compliance	`<regulation>_<type>_<retention>`	`hipaa_logs_7y`	Regulatory requirements

Detailed Examples

Bucket Name	Provider	Data Type	Retention	Org Unit
`aws_logs_35d_platform`	AWS	logs	35 days	Platform team
`azure_spans_14d_finance`	Azure	spans	14 days	Finance
`gcp_spans_14d_checkout`	GCP	spans	14 days	Checkout service
`audit_logs_365d_compliance`	Any	logs	365 days	Compliance
`debug_logs_7d`	Any	logs	7 days	Development

Naming Rules Reminder

Rule	Requirement
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot start with `default_` or `dt_`
Immutability	Names cannot be changed after creation

> **Sources:** Bucket management SDK (Dynatrace Developer) — *"Bucket name can not start with"* `default_` or `dt_`.

Retention Strategy

Retention Period Guidelines

Use Case	Recommended Retention	Rationale
Debug/Development	3-7 days	Short-term troubleshooting, high volume
Standard Operations	35 days	Monthly trends, incident response
Performance Analysis	60-90 days	Quarterly comparisons

Use Case	Recommended Retention	Rationale
Compliance/Audit	365-3657 days	Regulatory requirements

Retention by Data Type

Data Type	Typical Retention	Notes
Debug logs	3-7 days	High volume, low long-term value
Application logs	35-90 days	Operational analysis
Audit logs	1-7 years	Compliance requirements
Metrics	`default_metrics`, ~15 months	Custom metric buckets with their own retention from SaaS 1.349 (see *Cost Attribution Strategy*); before that, fixed
Spans	10-35 days	APM troubleshooting
Security events	90-365 days	Security investigations

Bucket Design Patterns

Pattern 1: Long-Term Compliance Data

Store critical audit information for extended periods:

```
Bucket: compliance_audit_logs
Table: logs
Retention: 1825 days (5 years)
Use case: SOX compliance, security audits
Route via: OpenPipeline filter on audit-level logs
```

Pattern 2: Short-Term Debug Logs

Reduce costs by storing verbose logs briefly:

```
Bucket: debug_logs_7d
Table: logs
Retention: 7 days
Use case: Development troubleshooting
Route via: OpenPipeline filter on loglevel=DEBUG
```

Pattern 3: Team Isolation

Separate buckets for team-based access:

```
Bucket: team_platform_logs
Table: logs
Retention: 35 days
Use case: Platform team owns infrastructure logs
Access: IAM policy restricts to platform team
```

Pattern 4: Application-Specific Retention

Business units with specific retention needs:

```
Bucket: finance_app_logs
Table: logs
Retention: 180 days
Use case: Financial application compliance
Cost allocation: dt.cost.costcenter set at the source (the bucket sets
retention, not chargeback)
```

Cost Control and Attribution

Cost Factors

Factor	Impact on Cost
Ingest volume (GiB/day)	Direct relationship
Retention period	Storage costs increase with longer retention
Query frequency	Query costs accumulate

Cost Attribution Strategy

Dynatrace cost allocation is field-based, not bucket-based: *"Tags applied at the OneAgent, Kubernetes, or telemetry-ingest layer flow through Grail as dt.cost.costcenter and dt.cost.product attributes"*, and enrichment *"is possible for all supported ingest methods, whether logs, traces, metrics, or events."* Before SaaS 1.349 metrics could not go into custom buckets at all; a cost field covers every signal on every tenant version. Use buckets for retention and access; use the cost fields for attribution.

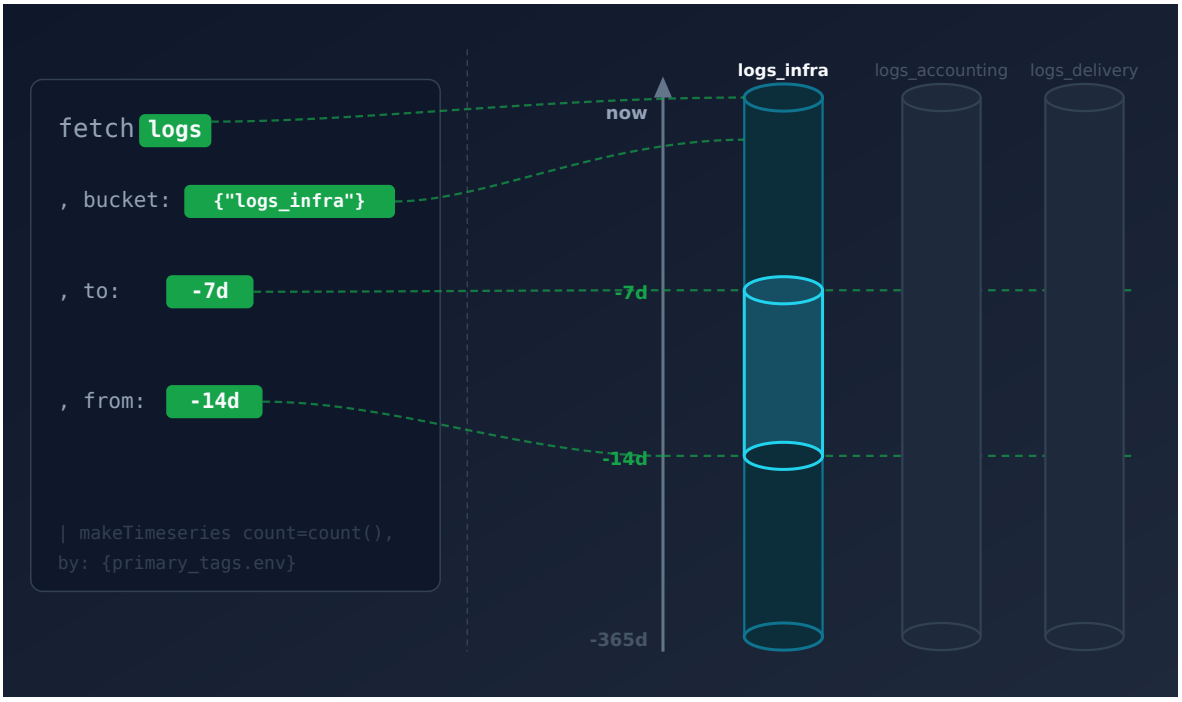
> **SaaS 1.349+ (rollout start 10/27/2026):** *"Metrics now supports custom buckets."* — *"You can optionally route subsets of metrics into custom buckets using configuration rules"* (What's new in Dynatrace SaaS 1.349 (DT docs)). The release lists cost control among the reasons; attribution still belongs on the cost fields, which work the same way for every signal.

Organization Level	Cost allocation	Bucket strategy
Business unit / cost center	`dt.cost.costcenter` at the source — host tag, `DT_TAGS`, Kubernetes namespace metadata, or OpenPipeline	A dedicated bucket only where the unit also needs its own retention or access
Product / application	`dt.cost.product`	A dedicated bucket above ~250 GB/day per application (*Query Performance at Scale*) or for distinct retention
Cloud account	Derive `dt.cost.costcenter` from the account field in OpenPipeline	Separate buckets only where retention or access differ

Reporting on allocated cost is covered in the FINOPS series.

> **Sources:** Set up Cost Allocation (DT docs) — as quoted, Configure cost allocation (DT docs) — *"map deployment-level primary Grail fields such as dt.host_group.id or host.name to dt.cost.costcenter and dt.cost.product via the Cost Allocation stage in OpenPipeline"*.

Analyzing Bucket Usage



DQL: Capacity Planning

Track ingest trends to validate routing rules and spot anomalies:

```
// Track log ingest trends by bucket over the last day — use for capacity
planning and routing anomaly detection
// For multi-day capacity planning use the billing-usage events in FINOPS-01
rather than a raw multi-day log scan
fetch logs, from:-24h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h),
dt.system.bucket}
| sort time_bucket desc
```

Bucket Strategy Considerations

Access Control Options

Buckets can serve as an access control mechanism:

Access Method	Description	When to Use
Security context (record-level)	Fine-grained ABAC at the record level	Complex multi-team scenarios
Bucket-level IAM policies	Restrict access to entire buckets	Simple team isolation
Combined approach	Security context + bucket policies	Maximum control and flexibility

Bucket-Based Team Isolation Example:

```
Team A: Access to team_a_logs bucket only
Team B: Access to team_b_logs bucket only
Platform: Access to all buckets
```

Query Performance at Scale

Bucket sizing shapes queryability because `fetch` stops after 500 GB of uncompressed data by default (override with `scanLimitGBytes`):

Bucket Ingest	Impact
One application below 250 GB/day	Can share a bucket — *"Applications below 250 GB/day can share a bucket without degrading query experience."*
One application above 250 GB/day	*"Create a dedicated bucket for any application ingesting more than 250 GB/day."* At 250 GB/day a query can reach back ~48 hours
1–3 TB/day per log bucket	Target range — ~4–12 hours of logs per query at the default scan limit
Above 3 TB/day (logs), above 2 TB/day (other tables)	Split — by line of business first, then by application where volume or compliance justify it

> **Docs differ (read 10/14/2026).** *Use Grail buckets to partition data* (updated 06/17/2026) says *"We recommend splitting up buckets that receive more than 2 TB of daily ingest." Best practices for a logs bucket strategy* (published 08/24/2026) is newer and specific to logs: *"Design your bucket strategy so that each bucket receives between 1 TB and 3 TB of log data per day."* This table follows the newer page for logs.

> **Sources:** Best practices for data partitioning (DT docs) — 250 GB rows as quoted, Best practices for a logs bucket strategy (DT docs) — *"Split log buckets by line of business rather than by individual application."*, Use Grail buckets to partition data (DT docs).

Data Immutability

Operation	Possible?	Alternative
Move data between buckets	No	Route new data via OpenPipeline
Delete selected records	Yes — record deletion API	Hard delete of the records a DQL filter matches, at most 24 hours of data per request; needs `storage:records:delete` (no default policy grants it)
Bucket deletion	Yes	Deletes ALL data permanently

> **Sources:** Record deletion in Grail via API (DT docs) — *"You can delete records from Grail using the API by leveraging DQL"*; *"The maximum timeframe for a record deletion request is 24 hours."*; *"There's no pre-defined policy that grants these permissions"*; *"Record deletion is final and can't be undone."*

Routing Data to Buckets

Buckets are assigned in OpenPipeline, in the **Storage** stage of a pipeline. That stage holds **Bucket assignment** processors — *"Assign records to the best-fit bucket."* — and the **No storage assignment** processor, and it runs **first match only**: *"Depending on the stage type, a processor returns either the first matching record or all matching records."* Each processor pairs a DQL matcher with a target bucket; the matcher *"defines the target of a processor via a DQL statement and narrows down the available data to the specific set you want to process."*

> **Corrected 09/24/2026.** Earlier versions of this notebook showed an OpenPipeline `processors: - type: route` YAML with `rules`, `destination` and `default`.

> OpenPipeline has no such processor or setting — configure Storage-stage bucket assignment as below.

Add one **Bucket assignment** processor per destination, most specific first:

Matching condition (DQL)	Bucket
`loglevel == "DEBUG"`	`debug_logs_7d`
`matchesPhrase(log.source, "audit")`	`audit_logs_365d`
`matchesValue(dt.host_group.id, "finance-*")`	`finance_app_logs`

Write matchers with the functions the OpenPipeline matcher documents — `==`, `matchesValue`, `matchesPhrase`, `isNull` / `isNotNull`, `iAny` — rather than `contains` or `starts-with`. `matchesValue` accepts a leading or trailing `*` and *"works with multi-value attributes (matching any value), and supports wildcards"*.

There is no `default:` route to set. Where no bucket assignment applies, the record is not lost: *"If you haven't specified bucket assignment in the pipeline configuration, Dynatrace will send all ingested data to the default buckets."* Check unmatched records in the pipeline preview before relying on that for a custom pipeline.

Routing on primary fields

OneAgent enriches telemetry at the source with standardized **primary fields** from the Semantic Dictionary (`dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`) plus customer-defined **primary tags**. They appear as top-level fields on metrics, spans, logs, business events and Smartscape entities, so bucket assignment can key on them directly (Latest Dynatrace):

Matching condition (DQL)	Bucket
`dt.cost.costcenter == "cc-1234"`	`finance_logs`
`matchesValue(dt.security_context, "pci")`	`pci_audit_logs_365d`
`dt.cost.product == "checkout"`	`checkout_logs`

Use `matchesValue` rather than `==` on `dt.security_context`: the field can hold an array, and the equality operator *"doesn't operate on elements being part of multi-value attributes."* See **ORGNZ-06: Security Context** for how the value is set and **OPLOGS** / **OPIPE** for pipeline configuration.

Logs from official Dynatrace database extensions already have a built-in bucket, `default_database_monitoring`, which keeps them out of `default_logs` for faster queries and tighter IAM scoping. The `default_` and `dt_` prefixes are reserved — a custom bucket name cannot start with either.

Smartscape Ownership integration. Smartscape entities now carry ownership information that Dynatrace Workflows can read directly — useful when bucket-routing decisions depend on which team owns the producing host or service. See **WFLOW** for the routing-on-ownership pattern.

> **Sources:** Processing in OpenPipeline (DT docs), DQL matcher in OpenPipeline (DT docs), Use Grail buckets to partition data (DT docs). Matchers executed as DQL filters on a live tenant 09/24/2026.

Bucket Design Checklist

Use this checklist when planning buckets:

- [ ] Defined naming convention aligned with organization
- [ ] Identified retention requirements by data type
- [ ] Set `dt.cost.costcenter` / `dt.cost.product` for cost attribution (buckets only where retention or access differ)
- [ ] Calculated expected ingest volumes (target 1–3 TB/day per log bucket; dedicated bucket per application above 250 GB/day)
- [ ] Planned for compliance requirements
- [ ] Documented bucket purposes
- [ ] Planned OpenPipeline routing rules
- [ ] Considered access control strategy (bucket vs security context)

Query Billing Model

Custom **log** buckets offer two query billing models:

Model	How Billed	Best For
Usage-based	Charged per GiB scanned at query time	Buckets queried infrequently or unpredictably

Model	How Billed	Best For
Retain with Included Queries	Queries over data inside the bucket's Included Queries period (10–35 days) are not charged separately; data older than that is queried usage-based	Buckets queried frequently or heavily by dashboards/alerts

"Customers can split a log bucket's retention period into two parts" — an Included Queries period of 10–35 days, inside an overall retention of up to 10 years. The choice is not permanent: *"Alternatively, if the Retain with Included Query option does not meet your use case and requirements, you can reconfigure a bucket at any time to use individually billed on-demand queries without losing data."*

> For compliance and audit buckets queried daily by security dashboards, **Retain with Included Queries** typically lowers total cost. For debug or ephemeral buckets queried only during incidents, **Usage-based** avoids paying for query capacity you rarely use.

Excluding Logs from Storage

When you extract metrics from logs via OpenPipeline (e.g., parsing error rates into a metric), you may not need to retain the source log records at all. Configure a **No Storage Assignment** in the OpenPipeline Storage stage to drop matching records after processing — this avoids retaining data purely for a signal you've already transformed.

> Use with caution: once records are dropped they cannot be recovered. Only use No Storage Assignment when metrics extraction fully captures the required signal.
>
> **Sources:** Log Analytics (DPS) (DT docs).

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-04**: Permissions in Grail Overview

References

- Grail Buckets
- Data Retention
- OpenPipeline Routing
- Log Analytics (DPS) (DT docs)
- Best practices for data partitioning (DT docs)
- Best practices for a logs bucket strategy (DT docs)
- Set up Cost Allocation (DT docs)

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.